

Finance Database Design and Implementation Report

Name : Ibrahim Abdulsalam

Student ID : 24127116

Introduction

The main aim of this project is to create a Relational Finance Database using SQLite. The database is a complete personal banking and investment management system, thus demonstrating what the structure of the real-world financial data would look like.

The primary objective of this project is to demonstrate:

- Relational database design
- Use of primary and foreign keys
- Implementation of multiple composite keys
- Enforcement of integrity constraints
- Realistic synthetic data generation including deliberate missing and duplicate data
- Ethical considerations in data simulation

Although the concepts we have been studying relate to database schema design, they can also be applied to the field of finance. The finance field is rich in interesting relationships between entities such as customers, accounts, transactions, and investments. As such, it is also a good domain to examine advanced techniques of schema design and relational modeling.

Database Purpose and Design Rationale

The main goal of this database is to create a basic financial system that will handle customer accounts, transactions, and investments in a simplified way.

The database consists of six interrelated tables:

- **Customers:** Stores personal customer information. This table has an entry for each unique customer of the financial institution.
- **Accounts :** An account classifies the type of financial account such as checking account, savings account or investment account. An account can have more than one owner. The Account class record is necessary to make a distinction between an account record and a customer record in order to allow for the possibility of joint accounts.
- **AccountHolders :** Many-to-Many Table in banking involve multiple account holders for a joint account and multiple accounts for a customer. The Account-holders table has a composite key of customer_id and account_id to uniquely represent these pairings.
- **Transactions:** This table holds all the account-level transactions. Every transaction is recorded in one row in this table with a unique transaction_id and a unique account_id.
- **TransactionCategories:** Categorises transactions into logical financial groups such as Income, Expense, and Transfer.
- **Investments:** Stores customer investment records including type, risk level, and purchase amount.

Each table was selected to represent a real-world financial entity.

Database Schema and Relationships

The database is a relational database with foreign keys enforcing referential integrity.

Relationships;

- Customers → AccountHolders (1:M)
- Accounts → AccountHolders (1:M)
- Accounts → Transactions (1:M)
- Transactions → TransactionCategories (M:1)
- Customers → Investments (1:M)

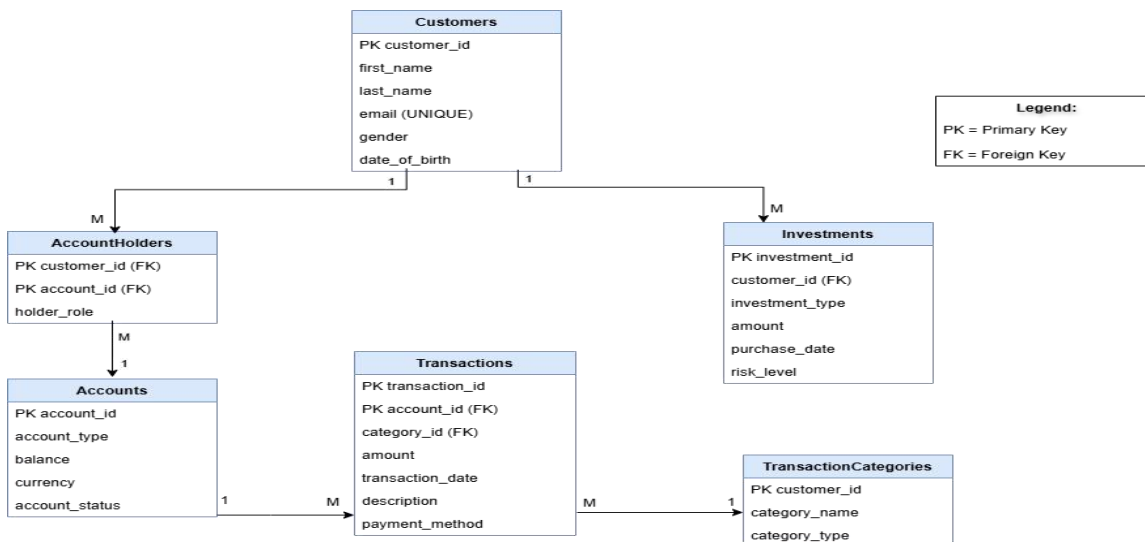
The AccountHolders table is needed to implement the many-to-many relationship between the Customers and Accounts tables. The AccountHolders table key is (customer_id, account_id).

A primary key is defined on the Transactions table with the name (transaction_id, account_id).

This design ensures:

- Referential integrity
- No orphan records
- Accurate modelling of joint accounts
- Prevention of duplicate relationship entries

Figure 1: ER Diagram of the Finance Database showing all tables, primary keys (PK), foreign keys (FK), and composite primary keys in AccountHolders and Transactions.



As shown in Figure 1, the schema includes multiple foreign key relationships and two composite primary keys, satisfying advanced relational design requirements.

Data Generation Methodology

The database was created with Python, using Google Colab and Faker library to generate random financial data.

The dataset includes:

- 500 Customers
- 700 Accounts
- Approximately 7,000–10,000 Transactions
- 400 Investments
- 5 Transaction Categories

Random distributions were applied to generate:

- Account balances between realistic financial ranges
- Transaction amounts between £50 and £5,000
- Dates distributed across the past two to three years
- Investment risk levels (Low, Medium, High)
- Multiple currencies (USD, EUR, GBP)

Deliberate Missing and Duplicate Data

This scenario will add some realism to the database. 3% of all customer email entries will be populated with a NULL value to simulate the missing data that is a part of real-world databases. The script will also add a few duplicate email entries to the database as well, to represent common data entry and database errors. This scenario is an example of the “Database Realism – Excellent” criteria where missing and duplicate data is intentionally added to the database.

Data Types and Constraints

The database incorporates a full range of data types:

- **Nominal:** gender, account_type, currency, risk_level

- **Ordinal:** account_status, risk_level
- **Interval:** transaction_date, purchase_date, date_of_birth
- **Ratio:** balance, amount

Key constraints include:

- PRIMARY KEY constraints on all core tables
- Composite primary keys in *AccountHolders* and *Transactions*
- FOREIGN KEY constraints across all relational links
- CHECK constraints on gender, account_type, account_status, payment_method, risk_level, and currency
- UNIQUE constraint on customer email

These constraints ensure structural integrity and enforce valid value ranges.

Ethical and Privacy Considerations

All data in this project is synthetic and has been generated using Faker library. There is no real personal or financial data in this project. The data distributions have been chosen to avoid any potential demographic bias and the data privacy guidelines that are required for good data engineering and data protection have been thoroughly adhered to.

Analytical Applications

The schema supports a wide range of financial analyses, including:

- Customer portfolio summaries
- Spending patterns by category
- Income vs. expense comparisons
- Investment risk distribution
- Transaction frequency analysis

The relational design also opens doors to new features such as fraud detection, credit scoring and predictive analytics.

Sample Data Verification

	transaction_id	account_id	category_id	amount	transaction_date	description	payment_method
1	1	1	1	45.78	2025-08-16	Environmental give church toward.	Card
2	2	1	3	3181.49	2025-03-16	Decision yeah ask no yourself month.	Transfer
3	3	1	3	850.75	2024-08-06	Score decision development your plac...	Card
4	4	1	1	186.7	2024-05-12	Laugh gas work.	Card
5	5	1	3	1903.51	2025-07-05	Continue finish image certain ...	Transfer

Figure 2: Sample of the Transactions table showing synthetic financial data and composite primary key (transaction_id + account_id).

As shown in Figure 2, each transaction is uniquely identified by the composite key (transaction_id, account_id), ensuring no duplicate transaction entries within an account.

Conclusion

The project delivers a fully realistic and ethical financial database in SQLite with 6 tables of high quality data for which all fields have the correct data type and constraint. The database is fully normalised, with all foreign and composite keys included, and follows the pattern of real financial systems.